

CUDA for Triton Programmers

Part I — The Main Topics

June 2026

Abstract

You already write GPU kernels — in Triton. So this course does not start from “what is a GPU.” It starts from the one idea that actually changes: **Triton hands you a tile; CUDA makes you build that tile out of individual threads.** Triton operates at the granularity of a *program* (one block, working on a block-sized tensor), and the compiler synthesizes the per-thread code, shared-memory staging, coalescing, vectorization, and pipelining for you. CUDA exposes that hidden layer and hands you the controls. This installment maps each Triton concept to its CUDA counterpart and names exactly what you now have to do by hand. Depth comes later — ask for any topic.

1 The one mental shift: program-level vs. thread-level

In Triton you write code from the point of view of *one program instance*: `pid = tl.program_id(0)` tells you which tile you own, and then you operate on whole vectors/tiles (`tl.load`, `tl.dot`, `tl.sum`). You never mention an individual thread. Under the hood each Triton program is compiled to a **CUDA thread block** of `num_warps`×32 threads, and the compiler decides how those threads cooperate to realize your tile operations.

CUDA removes that abstraction. A **kernel** runs as a grid of **blocks** of **threads**, and *you* write what a single thread does. The tile-level behavior you got for free in Triton is now something you compose explicitly from threads, warps, and shared memory.

Triton program $\equiv$ CUDA thread block.

A Triton tile op (load/dot/sum over `BLOCK_SIZE`) becomes “what do all the threads in this block do together.”

2 Concept map: Triton $\rightarrow$ CUDA

3 Vector add: same kernel, both languages

The Triton version you know:

```
1 @triton.jit
2 def add_kernel(x_ptr, y_ptr, out_ptr, n, BLOCK_SIZE: tl.constexpr):
3     pid = tl.program_id(0)
4     offs = pid * BLOCK_SIZE + tl.arange(0, BLOCK_SIZE) # a whole tile of indices
5     mask = offs < n
6     x = tl.load(x_ptr + offs, mask=mask) # tile load, masked
7     y = tl.load(y_ptr + offs, mask=mask)
8     tl.store(out_ptr + offs, x + y, mask=mask) # tile store
```

Triton	CUDA equivalent / what becomes your job
<code>tl.program_id(0)</code>	<code>blockIdx.x</code>
A program instance	A thread <i>block</i> (<code>blockDim = num_warps × 32</code> threads)
<code>BLOCK_SIZE</code> (constexpr tile)	A tile you cover with threads + loops; size is yours to pick
<code>tl.arange(0, BS) + offsets</code>	Per-thread index <code>blockIdx.x * blockDim.x + threadIdx.x</code>
<code>tl.load(p+off, mask=...)</code>	Manual global load + boundary if guard; <i>you</i> must make it coalesced
<code>tl.store(p+off, x, mask)</code>	Manual global store + guard
Implicit scratch for dot/reduce	Explicit <code>__shared__</code> arrays you allocate and index
Barriers inserted for you	Explicit <code>__syncthreads()</code>
<code>tl.dot(a, b)</code>	Tensor-core MMA via <code>wmma/mma</code> PTX or, better, CUTLASS/cuBLAS
<code>tl.sum</code> / <code>tl.max</code> (tile reduce)	Hand-written shared-memory tree reduction + warp shuffles
<code>tl.atomic_add</code>	<code>atomicAdd</code>
<code>num_warps</code>	<code>blockDim/32</code> — you set block size directly
<code>num_stages</code> (auto pipelining)	Manual double/multi-buffering with <code>cp.async</code>
<code>@triton.autotune</code>	Manual launch-config + occupancy tuning (or your own sweep)
Coalescing/vectorization (automatic)	Your responsibility: access layout, <code>float4</code> loads
Bank-conflict-free swizzling (automatic)	Your responsibility: pad/swizzle <code>__shared__</code>

Table 1: Read this table once; the rest of the document expands the interesting rows.

The CUDA version — note the tile (`tl.arange`) collapses to *one* index per thread, and the `mask` becomes an `if`:

```

1 __global__ void add_kernel(const float* x, const float* y, float* out, int n) {
2     int i = blockIdx.x * blockDim.x + threadIdx.x; // one element, not a tile
3     if (i < n)                                     // the "mask"
4         out[i] = x[i] + y[i];
5 }
6 // launch: blocks = (n + 255)/256; add_kernel<<<blocks, 256>>>(x, y, out, n);

```

Key differences in mindset: in Triton one *program* processed `BLOCK_SIZE` elements as a vector; in CUDA one *thread* processes one element, and `BLOCK_SIZE` elements are processed by `BLOCK_SIZE` threads running concurrently. The launch syntax `<<<blocks, threads>>>` plays the role of Triton’s grid lambda plus `num_warps`.

4 The layer Triton hid, part 1: threads and warps

Threads in a block execute in hardware groups of **32, a warp**, in lock-step (SIMT). Triton hid this; CUDA exposes it, and two things now matter:

- **Warp divergence.** If threads in a warp take different `if/else` branches, both paths run serially. (Triton’s masking compiled to predication that you didn’t have to reason about.) Keep neighboring threads on the same control path.
- **Warp-level primitives.** You gain direct access to fast intra-warp communication — `__shfl_down_sync`,

`ballot`, `vote`, cooperative groups — which Triton used internally for reductions. These let 32 threads exchange registers without touching shared memory.

5 The layer Triton hid, part 2: explicit shared memory

When you wrote `tl.dot` or `tl.sum`, Triton silently allocated on-chip scratch, staged tiles into it, inserted barriers, and chose a conflict-free layout. In CUDA this is manual and is the heart of most kernels:

- Declare it: `__shared__ float tile[TS][TS];`
- Cooperatively load global $\rightarrow$ shared (each thread loads a piece), then `__syncthreads()` before anyone reads.
- Mind **bank conflicts**: shared memory has 32 banks; if threads in a warp hit the same bank with different addresses, accesses serialize. The usual fix is padding (`[TS][TS+1]`) or swizzling — which Triton did for you.

6 Tiled matmul: where the abstraction gap is largest

In Triton, a matmul is essentially: loop over k in steps of `BLOCK_K`, `tl.load` an A tile and a B tile, `acc += tl.dot(a, b)`. Three lines hide an enormous amount. The CUDA equivalent makes each hidden step explicit:

1. Each block owns an output tile; each thread owns one (or several) output elements held in **registers** (the accumulator).
2. For each k -slice: cooperatively stage an A sub-tile and a B sub-tile from global into `__shared__`; `__syncthreads()`.
3. Each thread multiplies the shared tiles into its register accumulator; `__syncthreads()` again before reloading.
4. After the k -loop, write the accumulators back to global memory.

The `tl.dot` itself, on modern GPUs, lowers to **Tensor Core** MMA instructions. In raw CUDA you reach those via the `wmma` API or inline `mma` PTX — but for real work you should usually call **CUTLASS** (template library exposing the same tiling you know) or **cuBLAS**, rather than hand-rolling. A good reason to drop to CUDA is a fused epilogue or data movement that Triton/cuBLAS can't express.

7 Software pipelining: `num_stages` by hand

Triton's `num_stages` turns your k -loop into a multi-stage pipeline: prefetch the next tiles with asynchronous copies while computing on the current ones, hiding global-memory latency. In CUDA you build this yourself with `cp.async` (asynchronous global $\rightarrow$ shared copies) and double/multi-buffered shared memory — issue the load for stage $k+1$, compute stage k , rotate buffers. This is exactly the transformation the autotuner was searching over when it picked `num_stages`; now it is your code.

8 Occupancy and launch config: `num_warps`/autotune by hand

Triton’s `num_warps`, `num_stages`, and `BLOCK_*` were autotuned. In CUDA the analogous knobs are explicit and coupled:

- **Block size** = your `num_warps`×32, set in the launch.
- **Registers per thread** and **shared memory per block** limit how many warps run concurrently per SM — this is **occupancy**. Bigger tiles use more of both, reducing occupancy; the sweet spot is a trade-off you (or a sweep) find. Triton’s autotuner was exploring this surface.
- Enough occupancy lets the GPU *hide latency* by switching to other ready warps — the fundamental performance mechanism.

9 Memory coalescing: now your responsibility

Triton chose vectorized, coalesced access patterns for your `tl.loads`. In CUDA, the rule you must enforce by hand: **consecutive threads should touch consecutive addresses**, so the hardware merges a warp’s 32 accesses into a few wide transactions. Thread $i \leftrightarrow$ element i is coalesced; a stride or transpose in the wrong place wastes most of your bandwidth. Vectorized loads (`float4`) further cut transaction count — again, automatic in Triton.

10 When to actually drop from Triton to CUDA

Stay in Triton	Drop to CUDA / CUTLASS
Elementwise, fused pointwise, softmax, layernorm, attention-style fused kernels	Warp-specialized / producer-consumer kernels Triton can’t schedule
Standard tiled matmul / blocked reductions	Exotic data movement, custom epilogues, persistent kernels
Fast iteration, autotuned, portable	Squeezing the last % via hand-tuned <code>cp.async</code> /MMA
Most of deep learning	Library-grade kernels, or features outside Triton’s IR

Table 2: Triton covers a lot; CUDA is for the long tail and the last mile.

11 Tooling — mostly the same, plus you can now read the SASS

Profilers are shared across both worlds: **Nsight Systems** for the timeline, **Nsight Compute** (`ncu`) for per-kernel analysis (occupancy, memory throughput, stalls), `compute-sanitizer` for races/OOB. A bonus of working in CUDA: you can inspect the generated **PTX/SASS** directly — the same way you might dump Triton’s IR/PTX to see what the compiler did, now for code you wrote.

12 Road map (geared to you)

Summary. You already own the hard part — tiling, grids, and thinking in blocks. Learning CUDA is mostly learning the *sub-tile layer* Triton abstracted: composing tiles from per-thread work,

Topic	Framed against Triton
Shared-memory tiled matmul	What <code>tl.dot</code> + the k -loop actually compile to
Parallel reduction	What <code>tl.sum</code> does: tree reduction + warp shuffles
<code>cp.async</code> pipelining	<code>num_stages</code> by hand; double buffering
Tensor Cores (wmma/mma)	The instruction <code>tl.dot</code> lowers to; CUTLASS path
Occupancy & register tiling	The surface the autotuner searched
Warp specialization	The thing you leave Triton <i>for</i>
Coalescing & bank conflicts	The layout decisions Triton made silently

Table 3: Tell me which to expand first.

managing shared memory and barriers explicitly, respecting warps (divergence, shuffles), enforcing coalesced access, and hand-writing the pipelining and MMA that `num_stages` and `tl.dot` gave you for free. Everything in the concept-map table is a thread to pull — pick one.